mail-manager — Comprehensive Completion Plan

Generated after Phase D stabilization. This document synthesises **all original goals**, the status of every delivered phase, and the complete roadmap of remaining work to reach the finished state.

PART 1 — What the Project Must Be

Core Vision (unchanged from inception)

mail-manager is a **local, open-source email-intelligence and personal-productivity engine** that transforms raw communication streams into structured, queryable, continuously-updated knowledge — entirely on self-hosted infrastructure with no paid cloud services.

Thirteen Core Objectives (original requirement set)

#	Objective	Status
1	Ingest historical + new emails from **Gmail** and **Microsoft 365 Outlook**	☑ Implemented
2	Convert emails to Markdown for readability and LLM-friendliness	☑ Implemented
3	Store all data (emails, summaries, tasks, topics, calendar events) in PostgreSQL	☑ Implemented
4	Use `pgvector` for embeddings and semantic search	☑ Implemented
5	Group emails by thread and semantic topic	☑ Implemented
6	Generate daily summaries (morning + evening)	☑ Implemented
7	Maintain evolving topic snapshots	☑ Implemented
8	Detect junk mail while surfacing potentially valuable promotions	☑ Implemented (LLM analysis: `is_junk`, `category`)
9	Extract action items and sync them bidirectionally with Google Tasks	☑ Implemented (Phase D)
10	Project management with task lists, subtasks, priorities, and due dates	☑ Implemented
11	Combined calendar view (Google Calendar + Outlook Calendar, read-only)	☑ Implemented
12	Produce and update Markdown-based living documents	☑ Implemented
13	Modern web UI for browsing insights and interacting with the system	☑ Implemented

Additional Goals Added During Planning

Goal	Status
RAG-powered AI chat (ask questions about your emails)	☑ Implemented (Phase D)
Dashboard with real-time email stats and pipeline health	☑ Implemented (Phase D)
Persistent user configuration (LLM model, embed model, toggles) via DB	☑ Implemented (Phase D)
Pipeline audit log for observability	☑ Implemented (Phase D)
HTML email rendering in the browser	☑ Implemented (Phase D)
Gmail label translation (IDs → human names)	☑ Implemented (Phase D)
Label/category filtering in email browser	☑ Implemented (Phase D)

PART 2 — Phase History and Status

Phase A — Copilot Config ☑ DONE

Scaffolded `.github/` with `copilot-instructions.md`, `instructions/` files, `skills/`, and `agents/`.

Phase B — Repository Scaffold ☑ DONE

All 8 microservices + BFF + frontend scaffolded with Docker Compose, `uv` projects, migrations, health checks.

Phase B-patch — Scope Expansion ☑ DONE

Multi-provider DB schema, `calendar-sync` service, enhanced task model, living docs updated.

Phase C1 — Ingestion Service ☑ DONE

Gmail + Outlook providers, OAuth 2.0 + MSAL, email→Markdown converter, asyncpg pool, Redis publish. **52 tests passing.**

Phase C2 — Preprocessing Service ☑ DONE

Text cleaning, Ollama `nomic-embed-text` embeddings, Redis pipeline. **17 tests passing.**

Phase C3 — LLM Analysis Service ☑ DONE

Email categorisation, urgency, sentiment, action-item extraction, junk classification via Ollama `llama3.1:8b`. **17 tests passing (unit), Docker test passing.**

Phase C4 — Topic Tracking Service ☑ DONE

3-stage topic assignment (name match → pgvector similarity → create new), CRUD endpoints, snapshot evolution.

Phase C5 — Summary Generation Service ☑ DONE

Daily morning/evening summaries via scheduler, diff-hash dedup, Ollama integration.

Phase C6 — Task Management Service ☑ DONE

Task CRUD with subtasks, task lists, priorities, due dates, email extraction from LLM action items.

Phase C6.5 — Calendar Sync Service ☑ DONE (logic + tests)

Google Calendar + Outlook Calendar providers, event upsert, delta sync. **44 tests passing** (added in test coverage phase).

Phase C7 — BFF Layer ☑ DONE

Full auth flow (bcrypt + TOTP 2FA + JWT), 61+ proxy endpoints, auth middleware. **33 tests passing.**

Phase C8 — Frontend ☑ DONE

9 views, 8 Pinia stores, 11 API modules, light/dark theming, HTML email rendering, label/category filters.

Phase D — Polish and Intelligence Layer ☑ DONE

RAG chat streaming, Google Tasks bidirectional sync, dashboard stats (`email_stats` view, `pipeline_events` table), settings DB wiring (`user_config` table), pipeline audit log. **Migrations 004 + 005 applied.**

PART 3 — Current State Snapshot (post-Phase D stabilisation)

Infrastructure

- All 12 Docker containers healthy (postgres, redis, ollama, bff, ingestion, preprocessing, llm-analysis, topic-tracking, summary-generation, task-management, calendar-sync, frontend)
- Migrations 001–005 applied; DB has 16 tables + `email_stats` view

Test Baseline (as of this stabilisation)

Service	Passing	Failing	Notes
BFF	33	0	JWT key-length warnings in test fixtures (cosmetic)
Ingestion	52	2	`test_sync/fetch_without_auth` fail — DB conn before auth check
Preprocessing	17	0	
LLM Analysis	17	0	AsyncMock coroutine warning (cosmetic)
Calendar Sync	44	0	
Topic Tracking	1	0	Only `test_docker.py` runs outside Docker; other tests hang on lifespan DB conn
Summary Generation	1	0	Same lifespan issue

Service	Passing	Failing	Notes
Task Management	1	0	Same lifespan issue
Total	**166**	**2**	2 failures are pre-existing, known root cause

Open PR

- **PR #20**: `feat/phase-d → main` — 9 commits (including Phase D features + all C-series fixes). Ready to merge.

Active Branch

- `feat/phase-d` — HEAD at `05943ba` (stabilisation commit, post-migration)

PART 4 — Remaining Work (Phase E+)

Ordered by priority and dependency. Each phase is independently branchable.

Phase E1 — Merge PR #20 to Main

Priority: Critical — must happen before any other Phase E work.

Tasks:

1. Ensure PR #20 CI passes (or confirm tests in local Docker baseline)
2. Merge via rebase-and-merge (linear history)
3. Delete `feat/phase-d` remote branch after merge
4. Pull `main` locally; verify `docker compose ps` still healthy

Git checkpoint: merge commit on `main`, tag `v0.4.0`

Phase E2 — Living Documents Update

Priority: High — docs are 1–2 phases behind reality.

Files to update:

`docs/FACTS.md`:

- Update test counts to reflect actual baseline (166 total, 2 known failures)
- Update Phase D status to DONE
- Add new tables: `user_config`, `pipeline_events`, `email_stats` view, `html_body` column on emails, `calendars` table
- Add Phase D features: RAG chat, Google Tasks sync, dashboard stats, settings DB
- Add known limitations section

`PROJECT_OVERVIEW.md`:

- Section 5 (Data Model): add `user_config`, `pipeline_events`, `calendars`, `html_body` on emails
- Section 4.3 (LLM Analysis): mention chat/RAG capability
- Section 4.6 (Task Management): confirm bidirectional Google Tasks sync is live
- Section 4.8 (BFF): mention settings, stats, chat endpoints
- Update phase status table in summary section

`docs/PLAN.md`:

- Mark Phase D as DONE
- Add Phase E summary section pointing to this document

Git checkpoint: `docs: update living documents to reflect Phase D complete`

Phase E3 — Immediate Bug Fixes

Priority: High — user-visible or test-breaking issues.

E3.1 — Fix `ingestion.ts fetchProvider` query params (frontend)

File: `frontend/src/api/ingestion.ts`

`fetchProvider` computes `params` but calls `api.post(path)` without passing them. `syncProvider` ignores its `maxResults` argument entirely.

Fix: `ApiClient.post()` needs a third optional `params` argument, OR call the ingestion BFF proxy via `GET` with query params.

Preferred fix — add `params` to `post()` in `client.ts`:

```
async post<T>(path: string, body?: unknown, params?: Record<string, string |
number | boolean | undefined>): Promise<T>
```

Then update callers:

```
export function syncProvider(provider: string, maxResults = 100):
Promise<SyncResponse> {
  return api.post<SyncResponse>(`/ingest/sync/${provider}`, undefined, {
max_results: maxResults })
}

export function fetchProvider(provider: string, maxResults = 500, pageToken?:
string): Promise<FetchResponse> {
  return api.post<FetchResponse>(`/ingest/fetch/${provider}`, undefined, {
    max_results: maxResults,
    ...(pageToken ? { page_token: pageToken } : {}),
  })
}
```

E3.2 — Fix SyntaxWarning in ingestion `repository.py` (Python)

File: `services/ingestion/src/ingestion/repository.py` line ~191

Invalid escape sequence `'\_'` in raw SQL string. Fix by using a raw string:

```
"(SELECT 1 FROM unnest(labels) AS l WHERE l LIKE 'Label\\_%%')"
# or
r"(SELECT 1 FROM unnest(labels) AS l WHERE l LIKE 'Label\_%')"
```

(Note: PostgreSQL uses `\_` to match a literal underscore — the Python raw string `r"..."` preserves it correctly.)

E3.3 — Fix MSAL deprecation warning in ingestion (Python)

File: `services/ingestion/src/ingestion/router.py`

`get_authorization_request_url()` is deprecated in MSAL. Migrate to `initiate_auth_code_flow()` + `acquire_token_by_auth_code_flow()` pattern. This is a breaking change in the auth callback flow — plan carefully.

Steps:

1. Replace `get_authorization_request_url()` with `initiate_auth_code_flow()` (returns state dict)
2. Store the auth flow state dict in the DB-backed token store (see E5.1)
3. Update callback to call `acquire_token_by_auth_code_flow(auth_flow, auth_response)`
4. Update `/ingest/auth/url/outlook` BFF proxy + frontend `getAuthUrl()`

Git checkpoint: `fix(ingestion): migrate MSAL to auth_code_flow pattern`

Phase E4 — Test Coverage Gap: Service Unit Tests

Priority: Medium — the three services with lifespan DB issues need proper test isolation.

Services affected: `topic-tracking`, `summary-generation`, `task-management`.

Root cause: All three services eagerly call `await get_pool()` in their lifespan, which tries to open a real asyncpg connection. When running outside Docker, `postgres` hostname doesn't resolve.

Fix pattern (apply to all three services):

In each service's `tests/conftest.py`, monkeypatch the pool before starting the ASGI client:

```python
import pytest
from unittest.mock import AsyncMock, MagicMock, patch
from httpx import ASGITransport, AsyncClient

@pytest.fixture
async def client():
```

```
    mock_pool = MagicMock()
    mock_pool.fetch = AsyncMock(return_value=[])
    mock_pool.fetchrow = AsyncMock(return_value=None)
    mock_pool.execute = AsyncMock(return_value=None)

    with patch("topic_tracking.repository._pool", mock_pool), \
         patch("topic_tracking.repository.get_pool",
AsyncMock(return_value=mock_pool)), \
         patch("topic_tracking.events.get_redis", AsyncMock()), \
         patch("topic_tracking.events.subscribe_analyzed_emails", AsyncMock()):
        async with AsyncClient(transport=ASGITransport(app=app),
base_url="http://test") as ac:
            yield ac
```

Each service needs its own variant. Also fix:

- **BFF test JWT key** — change conftest fixture JWT secret to 32+ bytes
- **LLM analysis AsyncMock warning** — fix the mock setup for `stream()` in `test_chat.py`
- **Ingestion `test_sync/fetch_without_auth`** — mock `get_pool()` so auth check happens before DB access

Target: All 8 services runnable with `uv run pytest` from local machine, no Docker required.

Git checkpoint: `test: fix lifespan mocking to allow out-of-Docker test runs`

Phase E5 — Security Hardening

Priority: Medium — prevents real security issues in production use.

E5.1 — Move in-memory token store to DB (ingestion)

File: `services/ingestion/src/ingestion/router.py`

The `_token_store: dict[str, OAuthTokens]` in-memory dict loses all OAuth state on restart.

Fix: Store OAuth tokens in the existing `connected_accounts` table (already has `access_token`, `refresh_token`, `token_expiry`). The callback should upsert a `connected_accounts` row instead of writing to `_token_store`.

E5.2 — Encrypt OAuth tokens at rest

File: `db/migrations/` + `services/ingestion/src/ingestion/account_repository.py`

Currently `access_token` and `refresh_token` are stored in plaintext in `connected_accounts`.

Fix: Use `cryptography` library (Fernet symmetric encryption) with a key from `SECRET_KEY` env var. Encrypt on write, decrypt on read in the account repository. Add migration to re-encrypt existing rows (or note this is N/A for fresh installs).

Git checkpoint: `feat(security): encrypt OAuth tokens at rest and persist token store to DB`

E5.3 — Harden JWT test secret

File: `services/bff/tests/conftest.py`

JWT test secret is 23 bytes (below HS256 minimum of 32). Update to a 32-byte test secret to eliminate the `InsecureKeyLengthWarning` in test output.

Phase E6 — Frontend Polish and Completeness

Priority: Medium — several UI gaps in the current frontend.

E6.1 — Add `stats` Pinia store

The dashboard currently fetches stats directly via `getDashboardStats()` on mount. A Pinia store with auto-refresh (polling every 60s) would centralize this and allow other views to show live stats.

File: `frontend/src/stores/stats.ts` (new)

E6.2 — Fix `calendar` store actions

The `CalendarView.vue` may not have full event loading / time-range filtering wired correctly.

Audit `frontend/src/stores/account.ts` — does it handle the calendar account case (provider=google/outlook for calendar vs email)?

E6.3 — Chat panel refinements

The `ChatPanel.vue` uses SSE streaming. Verify:

- SSE disconnect handling (user navigates away mid-stream)
- Error display when upstream returns non-200
- Scope selector (global / topic / email) is wired to the chat request body

E6.4 — Settings view wiring

`SettingsView.vue` should load current settings on mount (GET `/api/v1/settings`) and persist each change (PATCH `/api/v1/settings/{key}`). Verify the `llm_model` dropdown actually calls the API and reflects the saved value.

E6.5 — Dashboard pipeline health display

The stats endpoint returns `pipeline` health keyed by stage. The Dashboard should display a "Pipeline Health" card showing last event time per stage (ingested, preprocessed, analyzed, topics-assigned, tasks-extracted). This makes pipeline problems immediately visible.

Git checkpoint: `feat(frontend): stats store, settings persistence, dashboard pipeline health`

Phase E7 — Frontend Testing

Priority: Low-Medium — currently zero frontend tests.

Setup:

1. Add `vitest` + `@vue/test-utils` + `@testing-library/vue` to `frontend/package.json`
2. Configure `vitest.config.ts`
3. Add `jsdom` as test environment

Test targets (priority order):

Target	Type	Value
`useMarkdown` composable	Unit	Verifies markdown rendering
`useToast` composable	Unit	Toast message queue logic
`auth` store	Unit	Login/logout/token handling
`email` store	Unit	Pagination, filter state
`LoginView.vue`	Component	Form validation, TOTP step

Git checkpoint: `test(frontend): add Vitest setup and initial store/composable tests`

Phase E8 — E2E / Pipeline Smoke Tests

Priority: Low — valuable for CI but not blocking core functionality.

Option A — Shell script smoke test: A `scripts/smoke-test.sh` that uses `curl` to:

1. Register a user and get a JWT
2. Trigger a Gmail sync (if credentials available) or POST a test email via the ingest API
3. Poll preprocessing, analysis, topic, task endpoints until data appears
4. Assert the chain completed in < 120s

Option B — pytest integration tests: A `tests/integration/` folder at the root with pytest tests that use real HTTP against the running Docker stack (requires `DATABASE_URL` + `REDIS_URL` to point at Docker ports).

Git checkpoint: `test: add pipeline smoke test script`

Phase E9 — Production Hardening

Priority: Low — only needed before real production deployment.

E9.1 — `make migrate` on Windows

The current `Makefile` calls bare `psql` which resolves to Git's bundled psql with incompatible path handling on Windows. Fix by wrapping migrations in docker exec:

```
migrate:
    docker compose exec -T postgres psql -U mailmanager -d mailmanager -f
```

```
/migrations/001_initial_schema.sql
    ...
```

Or simpler: a PowerShell script `scripts/migrate.ps1` that uses `docker compose cp` + `docker compose exec`.

E9.2 — Consistent error response shape

All services should return `{"detail": "message"}` for errors. Audit all services for places that return different shapes (plain strings, lists, etc.).

E9.3 — Rate limiting and pagination on BFF

BFF currently proxies without rate limiting. For production: add `slowapi` rate limiting middleware on the BFF, and ensure all list endpoints have pagination (limit/offset or cursor).

E9.4 — Structured logging completeness

Ensure all services log at key decision points with structured context (user_id, email_id, provider, etc.). Structured logs are important for operability.

PART 5 — Future Extensions (Phase F+)

These are out of scope for the current plan but defined in the original vision:

Extension	Effort	Prerequisites
Notion integration — push living docs + tasks to Notion pages	Medium	Phase E complete
Slack integration — ingest messages, surface in topics	Large	Add `slack` provider to ingestion service
Microsoft Teams integration	Large	MSAL Graph API scopes
Knowledge graph — entity graph (people, orgs, projects) across all data	Large	Separate graph service or Neo4j
Calendar write operations — create/edit events from UI	Medium	OAuth scope expansion
Multi-agent orchestration — specialised LLM agents for research/drafting	Large	Phase E + knowledge graph
Chrome extension — surface mail-manager insights in Gmail	Medium	BFF CORS + extension manifest
CI/CD pipeline — GitHub Actions for tests + Docker builds	Small	Phase E7 (tests)
Multi-user support — per-user auth, tenancy	Large	Schema changes + auth revamp

---

PART 6 — Execution Order Summary

```
E1 (Merge PR)
    └── E2 (Docs update)
        └── E3 (Bug fixes: ingestion escape, client.ts params) ← independent, can
do anytime
            ├── E4 (Test coverage: lifespan mocking)
            │   └── E7 (Frontend tests)
            ├── E5 (Security: token encryption + MSAL fix)
            ├── E6 (Frontend polish)
            └── E8 (Smoke tests)
                └── E9 (Production hardening)
                    └── F+ (Future extensions)
```

Phases E3, E5, E6 are independent and can be done in parallel on separate branches.

PART 7 — Definition of Done

The project is considered **complete** (at the originally-planned scope) when:

- ☐ PR #20 merged to `main`
- ☐ Living docs reflect current reality
- ☐ All 8 services pass `uv run pytest` locally (no Docker required for unit tests)
- ☐ Frontend tests exist and pass
- ☐ OAuth tokens encrypted at rest
- ☐ In-memory token store replaced by DB-backed store
- ☐ MSAL deprecation warning resolved
- ☐ `client.ts post()` supports query params
- ☐ Dashboard pipeline health cards working end-to-end
- ☐ Settings persist across restarts (already implemented; needs frontend verification)
- ☐ RAG chat works end-to-end with real emails

This document is the canonical completion plan. Update section 2 phase status as work completes. For each phase, open a feature branch (`feat/phase-e1`, `fix/e3-ingestion-warnings`, etc.) and submit a PR to `main`.